

Mind the Middle Layer: A Requirements-Driven Study of Wrong Mental Models in LLM-Agent-Controlled Robots

Md Akib Haider*

Department of Computer Science and Engineering
Islamic University of Technology, Bangladesh

Sakib Ahmed Shanto*

Department of Computer Science and Engineering
Islamic University of Technology, Bangladesh

Nafis Fuad Shahid*

Department of Computer Science and Engineering
Islamic University of Technology, Bangladesh

Dr Hasan Mahmud

Department of Computer Science and Engineering
Islamic University of Technology, Bangladesh

ABSTRACT

People can now tell a robot what to do in plain language. Behind that request, a large language model (LLM) agent interprets it, writes microcontroller code for the specific robot, checks the code, and flashes it to the board. The user sees none of this. They speak, and the robot moves, or it does not. We argue that this hidden middle layer leads to wrong mental models. People place intelligence in the robot instead of the agent, blame the wrong part of the pipeline when something fails, and misjudge what the robot can safely do. Because the robot acts in the physical world, these mistakes matter more than they do for a chatbot.

This is a framework-and-plan paper, not an empirical study. We contribute a stakeholder and requirements analysis; a plug-and-play architecture in which a connection handshake registers the robot’s capabilities and a validation stage runs before any flash; a taxonomy of seven wrong mental models drawn from the literature; a planned mixed-methods study of about one hundred participants to test that taxonomy; and mitigations tied to human–AI interaction guidelines, with an evaluation plan. We also built RoverLens, a simulator, and a scenario-based survey that tells participants where their understanding was wrong. The survey comes with an answer key, a catalogue of concrete cases for each wrong model, and a frame for reporting response statistics. No survey responses have been collected yet, and every participant example in the paper is hypothetical.

CCS CONCEPTS

• **Human-centered computing** → **Human computer interaction (HCI)**; *HCI design and evaluation methods*; • **Computer systems organization** → *Embedded and cyber-physical systems*.

KEYWORDS

mental models; folk theories; human–robot interaction; large language models; LLM agents; natural-language robot programming; code generation; trust calibration; explainability; requirements engineering

1 INTRODUCTION

Controlling a robot used to mean writing firmware. Now a growing number of systems let a person type or speak a command to an LLM agent, which works out the intent, writes control code for that particular robot, and runs it on the robot’s microcontroller [1,

36, 51, 55]. From the outside the exchange is short: the user talks, there is a pause, and the robot acts. Inside, several steps happen that nobody sees. The agent interprets the request, writes a program, validates it, compiles it, and flashes it to the hardware.

That invisibility is the design problem. Norman’s framing is useful here: a system works when its *system image* helps people cross the *gulf of execution* (what can I do?) and the *gulf of evaluation* (what just happened?) [26, 44]. A pipeline that hides its middle layer widens the second gulf. When the robot does something unexpected, the user has nothing to look at that would point to a cause. A mental model is a person’s own account of how a system works. It is usually incomplete and often wrong, yet people act on it and set their expectations by it [21, 27, 29, 45]. When the model is wrong, their actions and expectations drift away from what the system does, and they rarely notice. The quality of a person’s model is linked to how well they work with an AI teammate [4], and conversational agents in particular show a documented gap between what people expect and what they get [39].

We study one person operating one small robot, a wheeled or simple manipulator platform driven by an ESP32- or Arduino-class board, through an integrated agent. The main users have little or no robotics or programming background. Hobbyists and expert developers serve as comparison groups. The setting is short command episodes, and some of them fail. Because the robot is physical, a wrong model is more than an annoyance. Misjudging what the robot will do, or why it failed, can lead to unsafe actions or to throwing away hardware that works. That sets this problem apart from earlier mental-model work on recommender and conversational systems, where the stakes are mostly informational [19–21].

1.1 Research Questions and Contribution

We ask four questions. **RQ1**: What wrong mental models do non-expert users form of an agent → code-generation → hardware pipeline, and where do the errors cluster? **RQ2**: How do interface choices (voice or text, visible or hidden code, latency, failure messages) shape those models? **RQ3**: Can a wrong model be detected from behavior plus targeted probes, and does a detected flaw predict later errors? **RQ4**: Which mitigations repair the model and improve appropriate reliance without overwhelming the user?

This is a framework-and-plan paper: a theoretical and artifact contribution in the sense of Wobbrock and Kientz [57], not an empirical one. It reports no collected data. Its contributions are a requirements analysis grounded in the literature (§2); a planned study

*These authors contributed equally to this work.

and analysis (§3–§4); a plug-and-play architecture with a validate-before-flash stage, drawn as two diagrams (§5); a taxonomy of seven wrong mental models (§6); mitigations and an evaluation plan (§7); and an implemented simulator and understanding survey, with an answer key, a catalogue of cases for each category, and a frame for reporting response statistics (§3.3, §4.1, §6.2, Appendices A–B). Because no study has been run yet, we introduce the failure modes in §2 as motivating observations and develop them fully as the taxonomy in §6. The study in §3–§4 is designed to test that taxonomy. *All personas and numbers in §6.1 are hypothetical worked examples, not findings, and the response tables in §4.1 are empty because no responses exist.*

2 REQUIREMENTS, STAKEHOLDERS AND PROBLEM SPACE

Requirements work starts by asking who is involved, what they do, and what they care about [49]. *Primary users* are non-expert operators who give commands and try to make sense of what happens. *Secondary users* are hobbyists and expert developers, who serve as comparison groups and may extend the system. *Deployers* (educators, lab supervisors, product integrators) install the agent on a robot and answer for its behavior. *Domain experts* in robotics and NLP inform the capability model. These groups want different things. A primary user wants predictability and an easy way to recover. A deployer also needs safety. An integrator has to manage the cost and latency of a remote model.

We keep a risk register of assumptions, each marked as backed by evidence, uncertain, or still to be tested. **A1**: users will credit the robot with intelligence instead of the hidden agent. **A2**: users cannot tell a hardware fault from a code-generation fault without help. **A3**: users will assume the system can do anything they can put into words. Earlier work found that people are often unaware of invisible computational layers and invent folk theories to explain what they can see [19, 20], which fits A1–A3. Even so, we treat them as hypotheses for §3 to test, not as facts about this system.

Each motivating observation becomes a requirement that names an actor, a condition, and a success criterion, and each is tied to the literature instead of to our own preferences. Functional requirements describe what the system does. Data requirements say what must be recorded. Quality requirements cover reliability, learnability, and safety. Organizational requirements cover deployment and governance. Table 1 lists the core set. Several follow directly from established human–AI guidelines: make clear what the system can do and why it acted, and support efficient correction and dismissal [3]. They also follow from the idea that automation should support *appropriate* reliance, since both too much and too little trust cause harm [35]. The failure surface behind R5–R8 is developed as a taxonomy in §6.

3 EXPERIMENTAL SETUP

We plan a mixed-methods study because no single method captures a mental model well. Models are inferred, not observed [21, 52]. Think-aloud during use shows reasoning as it happens [18]. A post-scenario interview probes beliefs that are more stable. A structured accuracy instrument gives a score that can be compared across people. Together they tell us which wrong model someone holds

(the qualitative data) and how groups differ (the instrument). We chose the methods with the research questions and participants’ wellbeing in mind [49]. *This section describes a plan. No data have been collected.*

3.1 Participants and Data Gathering

We aim for about $n = 100$ participants, recruited through university and community channels and split into three background strata of roughly equal size: (G1) no computing or robotics background, (G2) hobbyists with some coding or maker experience, and (G3) CS or robotics experts. The stratification is the point of the design. Our main hypothesis is that wrong models differ systematically with prior exposure, so background is a variable of interest, not noise. About thirty people per stratum is enough for the planned comparisons of category incidence and still feasible for think-aloud sessions. It does not make the sample representative of any population. We will report attrition and any channel bias, since both limit what we can claim.

3.2 Procedures and Data Management

Each session takes roughly 45 to 60 minutes and has five steps. (1) A baseline questionnaire covers prior experience with robots, voice assistants, and AI, and the participant’s preferred language (English or Bangla), which lets us characterize the strata. (2) Participants work through *scripted fault-injection scenarios*, one for each taxonomy category in §6, in which the system fails in a controlled and pre-logged way: a disabled microphone, a request beyond the robot’s capabilities, an injected code-generation error, a context-window overflow. (3) They think aloud throughout. (4) A semi-structured interview follows the scenarios. (5) A mental-model-accuracy instrument combines a short prediction task (predict the robot’s behavior and the cause of a failure) with Likert items on what the person believes about the system. The RoverLens survey in §3.3 is a first version of this step.

The study needs ethics-board (IRB) approval before it runs. Participation is voluntary with informed consent, and people can skip items or withdraw at any point without penalty. Sessions are pseudonymized, and we keep only the data the analysis needs, under access control (R4 covers system-side logs, not participant identifiers). Fault injection is scripted and logged so that, in analysis, an observed failure can be tied to its true cause. Without that link we could not separate a genuine wrong mental model from a sensible reaction to a real fault (§8.1). One or two pilot sessions will refine scenario wording and order before the main run.

3.3 RoverLens Simulator and Survey

Status. This subsection describes apparatus that already exists. It reports no results. We have built a browser simulator, RoverLens, and an understanding survey to go with it, but no participant has used either one, so every response table in §4.1 is empty. The paper remains a framework-and-plan contribution.

The simulator. RoverLens simulates the pipeline in Figures 1 and 2 from start to finish. A deterministic parser stands in for the LLM. Command handling, validation, flashing, and robot motion are all simulated, so nothing needs hardware, an account, or a network connection. The simulator runs the six stages (1 input, 2 capability

Table 1: Core requirements, each traced to evidence or literature. IDs are stable and referenced from the design (§5), taxonomy (§6), and mitigations (§7).

ID	Type	Requirement	Evidence / rationale	Priority
R1	Functional	On robot connect, the system shall auto-register the device’s identity and a machine-readable capability manifest (sensors, actuators, board type) without user configuration.	Plug-and-play lowers the barrier for non-experts; grounds later refusal (A3).	Must
R2	Functional	The agent shall refuse, with an explanation, any command outside the registered capability manifest, rather than attempting a partial or fabricated action.	Ungrounded LLMs otherwise produce unsafe or nonsensical actions [1]; “make clear what the system can do” [3].	Must
R3	Quality (safety)	No generated code shall reach the microcontroller before passing static, dry-run, and safety validation.	A physical effector makes unchecked code a safety hazard.	Must
R4	Data	The system shall retain per-episode provenance (utterance, generated code, validation result, flash status, execution feedback) to support diagnosis and study analysis.	Enables distinguishing user confusion from genuine system failure (§8.2).	Must
R5	Quality	The interface shall report failures with a distinguishable, layer-specific cause (input, capability, code, deployment, execution).	Users cannot otherwise localize faults (A2); supports appropriate reliance [35].	Must
R6	Quality	The interface shall make the hidden pipeline legible on request (e.g. “generating code,” “uploading,” “running”).	Revealing an invisible layer reshapes users’ models [20]; “make clear why the system did what it did” [3].	Should
R7	Quality	The system shall expose the active session/context state so a user can see when earlier commands are no longer available.	Context-window limits silently break follow-ups [38].	Should
R8	Organizational	When the underlying model or firmware changes, the deployer shall be able to surface a change notice to users.	Silent change violates expectations and triggers resistance [12]; “notify users about changes” [3].	Could

check, 3 agent, 4 code validation, 5 deployment, 6 execution and feedback), and a facilitator can inject a controlled fault at exactly one of them: a generated-code error, a missing camera, an expired context, an unavailable agent while the board stays connected, or a firmware update that cuts the maximum motion time from 5 s to 3 s. There are two interface conditions. They share the parser, the faults, the timing, and the animation, and differ only in what they disclose. The opaque baseline (A) shows generic progress and generic errors. The transparent condition (B) makes the requirements in Table 1 visible (Table 2). Each episode is logged with the command, generated code, checks, flash state, feedback, injected fault, true cause, and firmware version (R4), so a failure can always be traced back to its cause.

The survey. After using RoverLens, a participant fills in a web survey: a short background form and ten scored parts (Table 3; the full questionnaire is Appendix A). The parts are a baseline command; four scenarios taken from Table 4 (WM2, no motion after a generated-code fault; WM3, a missing camera; WM4, a follow-up after the context has expired; WM6, an unavailable agent while the board stays connected); a simulated firmware update (WM7); a retry after the fix, which checks whether people think the robot learned something; two short vignettes that need no simulator (WM5, trust after early success; WM1, an English/Bangla contrast); and a closing part about the whole system with six Likert items. Each scenario part asks one single-choice question about what happened (marked *core*), a set of true/false statements, an open explanation, and a 1–5 confidence rating. Participants can skip any part. The wrong options in the closed questions are written from the misconceptions we expect, as in concept inventories and diagnostic tests [24, 53].

Table 2: Requirements as implemented in RoverLens, and what each interface condition discloses.

ID	Simulated behavior	A opaque	B transparent
R1	Handshake registers the board and its capability manifest	hidden	shown
R2	Requests outside the manifest stop before code generation, with no partial action	refused, no reason	refused, with reason
R3	Static, dry-run and safety checks gate deployment	enforced	enforced, shown
R4	Per-episode provenance exported as JSON	logged	logged
R5	Layer-specific failure messages	generic error	names the layer
R6	Six visible stages and a technical trace	generic progress	shown
R7	Last available action; context expiry marked	hidden	shown
R8	Optional firmware notice (limit 5 s → 3 s)	hidden	shown

Options are shuffled for each participant, “not sure” is always last, and nothing is marked until the end, so feedback on one part cannot change the answers to the next. At the end the participant sees, for each part, whether their understanding was correct, partly correct, or incorrect, what really happened, and which category of wrong model a mistaken answer matches.

Table 3: Structure of the understanding survey. Points come from the closed items: one for each single-choice or multi-select question and one for each true/false or placement row. Open, Likert, confidence, and background items are not scored.

Tag	Part	Mode	Choice	Grids	Open	Pts	Targets
ABOUT	About you and your session	form	0	0	0	–	background
00	A simple command	simulator task	1	1	1	4	baseline
WM2	The rover did not move	simulator task	2	1	1	7	WM2
WM3	Find the red ball	simulator task	1	1	1	6	WM3
WM4	“Do that again”	simulator task	2	1	1	7	WM4
WM6	Who does the thinking?	simulator task	2	1	1	6	WM6
WM7	It used to work	simulator task	1	1	1	5	WM7
LEARN	After the fix	simulator task	1	1	1	5	LS (candidate)
WM5	Vignette: early success	vignette	1	1	0	4	WM5
WM1	Vignette: two languages	vignette	1	1	0	3	WM1
OVERALL	Putting it together	post-use	0	2	2	9	WM6, WM4, WM2, LS
Total						56	

Any simulator or vignette part can be skipped. Skipped parts are left out of scoring; they do not count as wrong.

Relation to the procedure above. The survey covers step (5) and part of step (2). Of the faults listed in step (2) it includes the request beyond the robot’s capabilities, the code-generation error, and context loss (as a forced expiry, not a real token overflow). It adds an unavailable agent and a firmware change. It has no microphone fault because the prototype is text-first. The survey captures beliefs after the fact, not reasoning as it happens, so its open answers only partly stand in for think-aloud. Completing every part takes about 20 to 25 minutes, which does not fit a 45 to 60 minute session unless parts are skipped or the survey is run remotely. It is in English only, while step (1) offers Bangla. Someone who uses both interfaces is influenced by the first one (seeing the transparent version can teach the pipeline), so we prefer between-subjects assignment, or counterbalanced order with the order reported.

Scoring and verdicts. Only closed items are scored. A single-choice or multi-select question is worth one point, and each true/false or placement row is worth one point. Multi-select credit is $\max\{0, (h-w)/r\}$, where h and w are the numbers of correct and incorrect options chosen and r is the number of correct options. “Not sure” earns zero but is not counted as a misconception. A part is judged *correct* if it earns at least 80% of its points and its core items average at least 75%. It is judged *incorrect* if its core items average below 50% or it earns below 50% overall, and *partly correct* otherwise. The overall verdict is “correct” only when the total is at least 80% and no part was judged incorrect. The core rule keeps easy true/false rows from hiding a wrong central idea. A confidence of 4 or 5 on a part that was partly correct or incorrect is flagged as over-confident, which is the over-trust signal behind WM5. Open answers are never scored. The participant sees them next to a model answer, a keyword match against listed key ideas is stored only as a hint, and the analysis of record is codebook coding as described in §4, for which we generate a coding sheet with blank columns for two coders. The answer key, with the category tag and the reason for every distractor, is Appendix B.

4 DATA ANALYSIS

Interview and think-aloud transcripts will be coded with a hybrid scheme. The deductive layer applies the seven taxonomy categories (§6) as a starting codebook, since the study exists to test that framework. The inductive layer follows reflexive thematic analysis [6], so new categories can appear and existing ones can be refined or split. We keep negative cases instead of discarding them. Every reported pattern will carry an evidence trail from raw segment to code, category, and design implication, with the linked system logs (R4) as support.

Coding reliability. Two coders will code the transcripts and the open survey answers independently against the codebook, each blind to the other’s codes and to the participant’s stratum and interface. We report Cohen’s κ [9] for the deductive categories and Krippendorff’s α [32] where a segment can carry several codes. Disagreements are resolved by discussion and logged. Agreement is a check on the codebook, not a score to maximize, and for interpretive coding such as the inductive layer, discussion and a documented audit trail matter more than a single coefficient [40]. Coefficients are reported with the number of coded segments.

4.1 Data Presentation and Response

Findings will appear as a filled-in version of the taxonomy table (§6): each category with its observed incidence per stratum, a representative anonymized quote, and the mitigation it motivates. That keeps the path from evidence to design easy to follow and to check.

Status. The tables in this subsection are generated by script from exported response files. They are empty because no responses exist, and nothing here is an empirical result. Their purpose is to fix in advance what we will report and how.

Method. The unit of analysis is the participant. Building on §4, proportions come with 95% Wilson score intervals [56]. Category incidence across the three background strata is tested with a Pearson χ^2 test [47], reported with Cramér’s V [10] and the denominators, and any row with an expected count below 5 is flagged for an exact

test. The difference between interfaces A and B in a proportion gets a Newcombe hybrid-score interval [43], and the difference in mean score gets a percentile bootstrap interval [16] (2000 resamples, fixed seed, so the tables can be reproduced). Because the plan is fixed before any data arrive, it can also be preregistered [8]. Incidence of a category is computed over participants who had a chance to show it, meaning at least one part they did not skip contains an item tagged with that category. With about 30 people per stratum, a Wilson interval for a proportion near one half is roughly ± 0.17 wide, so differences of that size between strata should not be over-read. We report estimates with intervals and effect sizes instead of relying on significance tests alone [14]. Incidence depends on these scenarios and on how the closed items were built, so it is not a population base rate. People who used both interfaces are reported separately, by order, and left out of the A-versus-B contrasts. That contrast bundles several disclosure changes, so it cannot attribute an effect to a single mitigation (§7). The tables these estimates fill (incidence of each category overall, by stratum and by interface, plus the sample, verdicts per part, fault localization, structural placement, confidence, ratings, overall score, open-answer coverage, and item-level accuracy) are generated by the same script from exported response files. They are not printed here because they would be empty.

5 METHOD

Existing work already covers the individual pieces of this pipeline, and we reuse them instead of reinventing them. End-user robot programming has long tried to lower the barrier to authoring robot behavior [2], and LLMs are the latest step. *Code as Policies* showed that code-writing LLMs can generate robot control programs from natural-language commands by composing perception outputs and control-primitive APIs [36]. *ProgPrompt* produces situated task plans as executable Python with precondition checks [51]. *SayCan* grounds language in what a robot can actually do, and points out that ungrounded LLMs otherwise interpret instructions in ways that are “nonsensical or unsafe” for the physical situation [1]. Practitioner guidance on combining a high-level function library with real robot APIs also exists [55]. On the embedded side, generating, compiling, and flashing Arduino/ESP32 firmware from natural language is already possible with open tools [5]. Generated code is also imperfect in ways that users tend to miss [7, 48, 54].

What this earlier work does not cover is what we add, and we want to say so plainly. First, we frame the problem around users’ *mental models* of the whole pipeline, with a taxonomy and a study to test it. Second, we propose a *plug-and-play capability handshake* that turns device registration into a manifest the agent can use to refuse gracefully. Third, we place a *validate-before-flash* safety stage between code generation and the hardware. The code generation, grounding, and flashing mechanisms themselves come from the cited systems.

Figure 1 shows the architecture, using the module names we use throughout the paper. The **Device/Peripheral layer** auto-detects the robot and runs a configuration handshake when it connects (similar to USB HID enumeration or OS driver binding), which produces a capability manifest (R1). The **Input/Perception layer** handles the microphone, camera, and sensors, and if a required

sensor is missing it reports that specific absence instead of carrying on (R5). The **Agent/Reasoning layer** parses intent, plans, and generates microcontroller code. It can run on a local, on-device (fine-tuned) model or on a remote API model. With a remote model, a **connectivity handler** manages retries, timeouts, an offline fallback, and a latency budget. The **Code Validation layer** runs static checks, a dry run, and safety checks before anything is flashed (R3). The **Deployment layer** compiles and flashes the microcontroller, allowing for toolchain differences such as ESP-IDF/Arduino-core for ESP32 versus AVR for classic Arduino. The **Execution & Feedback layer** confirms whether the action happened and reports back to the agent and the user (R5, R6).

Figure 2 follows a single command at runtime. Every way the episode can fail leaves through a different, labeled report (missing capability, invalid code, deployment failure, execution failure), so the interface can name the cause (R5). This is the design counterpart of the category on undiagnosed fault sources (§6, WM2): the system tells fault types apart precisely because users, on their own, cannot.

6 WRONG MENTAL MODEL TAXONOMY

We derived the taxonomy deductively from three bodies of work: mental-model and folk-theory research on invisible computational layers [12, 19–21], research on trust calibration in automation [35], and the known limits of LLM-driven robots [1, 38]. We then mapped it onto the failure surface of the architecture in §5. Four categories come from our motivating observations (WM1–WM4) and three were added from the literature (WM5–WM7). For each one we give the cause, a concrete failure scenario, and the reason a user would misattribute it. Table 4 summarizes the set and links each category to a mitigation (§7).

WM1: uneven language or code competence read as “the robot is broken” or “it got dumber.” Model performance is uneven across input languages and target boards. LLMs do markedly worse in lower-resource languages than in high-resource ones [28], and code quality can differ across microcontroller toolchains. Picture the same instruction working in English but degrading in Bangla, or working on an Arduino Uno but not on an ESP32 variant. The user cannot see the language- or board-specific code path, so the difference gets blamed on the robot (“broken,” “dumber today”) instead of on gaps in the hidden generator.

WM2: an undiagnosed fault treated as a single, permanent failure. A failed command can start in the microphone, the motors, the generated code, or a thermal shutdown. Each needs a different fix, and none is permanent. Say the robot does not move. The real cause is an injected code-generation error, the microphone light was on, and the motors are fine. Without layer-specific feedback the user folds these distinct faults into one verdict (“it doesn’t work”) and may throw away hardware that works. The grounding literature notes that ungrounded generation is itself a common and non-obvious source of failure [1], and appropriate reliance requires that users can tell capable states from incapable ones [35]. Trust in a robot drops with observed failures and depends on the feedback it gives about them [11, 50], and HRI already has a vocabulary for failure types and how to resolve them [25].

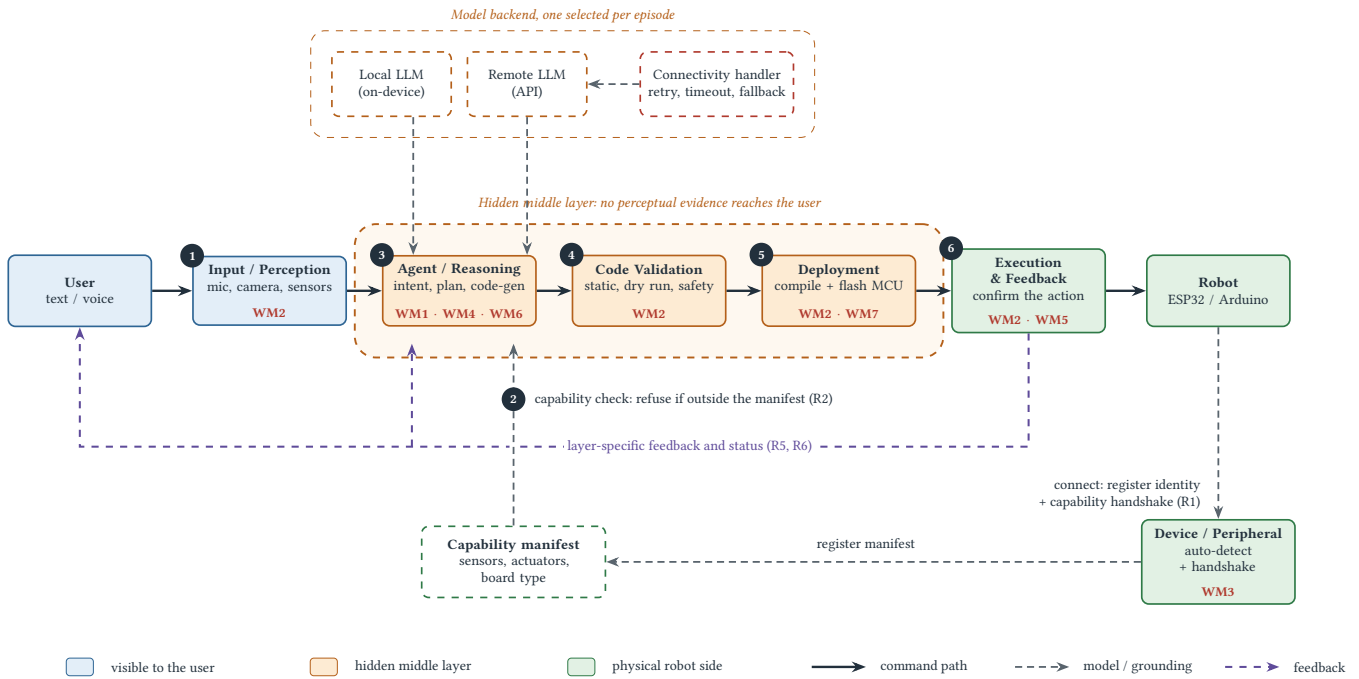


Figure 1: System architecture. Blue nodes are what the user can perceive, orange nodes make up the hidden middle layer, and green nodes sit on the robot side. Numbers 1–6 are the pipeline stages used throughout the paper; stage 2, the capability check, runs against the manifest. Red tags mark the wrong-mental-model categories (§6) that can arise at each module. Module names match §5 and Table 1.

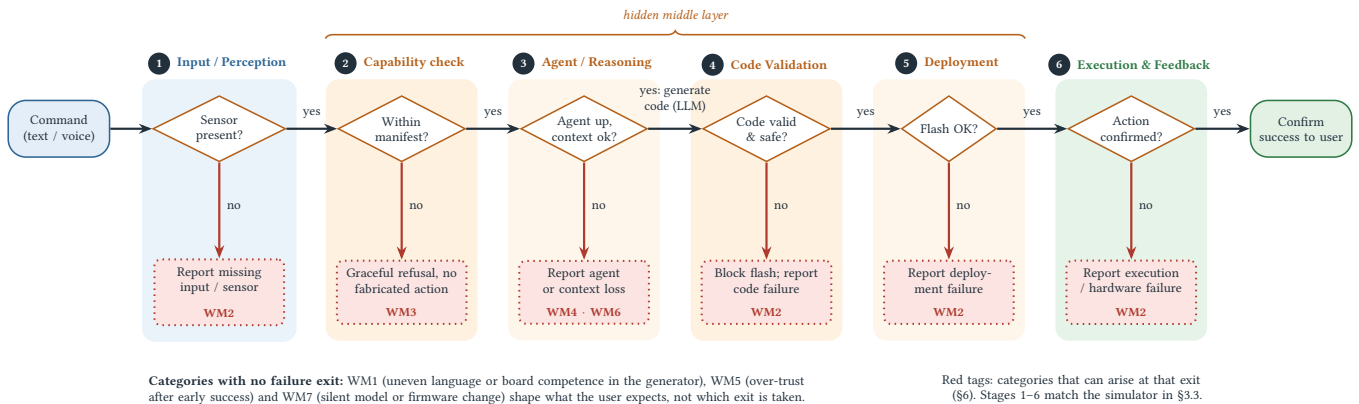


Figure 2: Operational flowchart for one command, drawn over the six pipeline stages. Each decision sends failure to its own labeled report, so the interface can name the cause (R5). This is the design counterpart of WM2 (§6). Blue: visible input; orange: hidden middle layer; green: execution on the robot.

WM3: no model of the robot’s actual capabilities. Without an exposed capability manifest, users assume that anything they can say, the robot can do. Suppose a command needs a sensor the robot does not have, and the system hallucinates or half-executes it instead of refusing. The user reads that partial behavior as “it tried and almost got it” and infers an ability that does not exist. SayCan describes this problem: ungrounded LLMs produce unsafe or nonsensical

actions when a request goes beyond real affordances [1]. Guideline G1, make clear what the system can do, addresses it directly [3].

WM4: context-window limits read as the robot “forgetting” or “ignoring.” A follow-up that refers to something from minutes earlier can fail because of context truncation, a mid-generation token limit, a remote API error, or high latency. These are different causes. Take “Do that again, but slower,” which fails because the earlier action has

fallen out of context. The user concludes that the robot is disobedient or has a bad memory, not that a context boundary was reached. Long-context models are known to use their context unevenly and to lose information in the middle [38], and that state stays invisible unless the interface shows it (R7).

WM5: trust miscalibration after early successes (added). A run of early successes inflates trust beyond the system's real reliability. After several flawless commands, a user hands over a higher-stakes action without checking, treating past success as a guarantee. This is the classic over-trust failure of automation [15, 46], and it is amplified here because users tend to miss faults in generated code [48, 54]. Robot performance is also the strongest influence on trust in robots [22]. Systems should be designed for appropriate reliance, because both over- and under-trust do harm [35], and calibrating trust is an explicit human-AI guideline [3].

WM6: too much agency and intent given to the robot (added). People respond socially to computers and attribute understanding and intention to them [42], and they read human-like agency into non-human things that behave in goal-directed ways [17]. Autonomy and transparency also change whom people blame when a robot errs [30]. A user who believes "the robot understood me and chose to move" has collapsed the hidden agent and code layers into a single intending robot. Intelligence gets placed in the effector instead of the generator, and that skews every later expectation. Earlier work shows that people build models of an AI from its observed behavior, not from the technology underneath [21].

WM7: version drift, "it used to work" (added). A silent update to the model or firmware changes behavior between sessions. A command that worked last week acts differently after an unannounced model swap. The user holds a stable model of what they assume is a static system, so they read the change as a malfunction or as their own mistake. Unannounced algorithmic change is known to cause confusion and resistance [12], and telling users about changes is a guideline (G18) [3] that R8 covers here.

6.1 Sample Illustration

The personas and the small analysis below are made up for illustration. They are not participants, not data, and not results from the planned study in §3–§4. Their only purpose is to check that the instrument could surface each category and to show how the coding scheme would be applied. The wording is hedged on purpose.

- **Rafi, 19, Bangla-first, no computing background.** On WM1, he *might* say the robot "stopped understanding" after he switched from English to Bangla. That would plausibly be WM1, and possibly WM6.
- **Nadia, 27, hobbyist maker.** Facing a silent no-move (WM2), she *could* guess "dead motor" when the injected fault was in the code, which would illustrate WM2.
- **Tanvir, 22, CS student.** Asked for an action that needs an absent sensor (WM3), he *might* accept a partial motion as "almost worked," plausibly WM3.
- **Mrs. Karim, 45, English-first, non-technical.** After a failed follow-up (WM4), she *could* say the robot "ignored" her, consistent with WM4.

- **Shuvo, 31, expert developer.** After several successes he *might* hand over a riskier command unchecked (WM5), and separately *might* remark that "it behaved differently than last week" after an unannounced update (WM7).

Worked example on synthetic data (not findings). If these five invented utterances were coded with the scheme from §4, two independent coders might assign WM1, WM2, WM3, WM4, and WM5/WM7 in turn. A disagreement over Rafi's utterance between WM1 and WM6 would be logged as a boundary case and settled by discussion, and it would lead to a codebook note separating "competence gap" attributions from "agency" attributions. The numbers only show the procedure. They are not incidence estimates and say nothing about a real population.

6.2 Wrong Mental Model Categories

A *case* is one specific wrong belief that a closed survey item can pick up: a tagged distractor that a participant selects, or a false statement that a participant accepts. Each case carries one category from Table 4, assigned by the item author. Table 5 shows the scenario and core item behind each category, and the catalogue below lists every case by category, with the number of participants who endorsed it once responses exist.

The tags follow the same boundary logic as the walkthrough in §6, as far as closed items allow. An option that gives the robot a will ("decided not to move") is coded WM6. "Forgot or ignored me" is coded WM4 and always follows a reference to an earlier turn. A broken-hardware explanation is coded WM2 in single-language scenarios, but WM1 in the language vignette. A change put down to wear is coded WM7, although the survey's WM7 is a change within one session, not a comparison across sessions, so its cases are better read as beliefs about silent change.

The catalogue also has one category that is not among WM1–WM7. **LS, assumed learning or adaptation**, is the belief that the robot learns, remembers across sessions, or improves with use. An example is thinking that a retry worked because "the rover learned from its failure." It came up when we asked what people think the system learns. We propose LS only as a candidate for the inductive layer in §4. If it never appears in coded open answers, or is always absorbed by WM4 or WM6, we will drop or merge it and say so. A chosen distractor shows a belief that fits a category, not that the participant reasoned that way. We therefore read closed-item incidence next to the coded open answers and do not merge the two.

WM1 — Uneven language competence read as "broken". Different performance across languages was attributed to broken hardware or the robot "getting dumber".

- **wm1_why:** chose "The robot's hardware is failing". Endorsed by –/–.
- **wm1_why:** chose "The robot has gotten worse over time". Endorsed by –/–.
- **wm1_grid/b1:** judged "The failure necessarily means the motors changed." to be true. Endorsed by –/–.

Table 4: Taxonomy of wrong mental models. Each category links to a design mitigation (§7) and its grounding literature. WM1–WM4 are the motivating set; WM5–WM7 are added from the literature.

ID	Wrong model	Underlying (true) cause	Why misattributed	Grounding
WM1	“Robot is broken / got dumber”	Language- or board-dependent code-gen competence gap	Hidden, language/board-specific code path invisible	[28]
WM2	One permanent failure	Distinct fault sources (mic, motor, code, thermal)	No layer-specific feedback	[1, 35]
WM3	“It can do anything I say”	Real capabilities bounded; no exposed manifest	Partial/fabricated action read as near-success	[1, 3]
WM4	“It forgot / ignored me”	Context truncation, token limit, API error, latency	Session/context state invisible	[38]
WM5	“It always works”	Reliability below inflated expectation	Early success read as guarantee	[3, 35]
WM6	“The robot understood and chose”	Intelligence is in the hidden agent, not the robot	Social/anthropomorphic response to machines	[21, 42]
WM7	“It used to work”	Silent model/firmware update changed behavior	Stable model of an assumed-static system	[3, 12]

Table 5: Where each category is tested in RoverLens and the survey. “True cause” is what the simulator actually did. Core items are the single-choice questions that decide a part’s verdict.

ID	Participant task	True cause in the simulator	Core items
WM1	Vignette only (RoverLens accepts English)	Language coverage of the generator, not hardware	wm1_why
WM2	<i>Where did it fail?</i> Send “Move forward for 2 seconds”	Generated code calls an undefined function; fails static validation; nothing flashed; motors fine	wm2_stage, wm2_cause
WM3	<i>A capability boundary.</i> “Find the red ball”	Manifest has no camera; refused at the capability check before any code is generated	wm3_can
WM4	<i>A missing memory.</i> Full command, then “Do that again”	Context expired before the follow-up; fails at the agent stage; no code generated	wm4_why, wm4_where
WM5	Vignette only (success on one command, delegation of another)	Success on one task says little about tasks that use other layers or capabilities	wm5_why
WM6	<i>Who does the thinking?</i> “Turn left”	External agent unavailable while the board stays connected; no new program	wm6_where, wm6_why
WM7	<i>It used to work.</i> A 4-second command before and after a simulated firmware update	Update lowers the motion limit from 5 s to 3 s; blocked at validation	wm7_why
LS	<i>After the fix.</i> Retry after the fault is removed	The cause was removed; nothing was learned	ls_why

WM2 — *Undiagnosed fault source.* Different failure causes (input, generated code, upload, motor feedback) were treated as one permanent hardware failure, or the failure was placed at the wrong layer.

- wm2_stage: chose “Your command never reached the system (input)”. Endorsed by –/–.
- wm2_stage: chose “The system decided the rover can’t do this (capability check)”. Endorsed by –/–.
- wm2_stage: chose “The AI agent couldn’t interpret the request (agent / reasoning)”. Endorsed by –/–.
- wm2_stage: chose “The program couldn’t be uploaded to the rover (deployment)”. Endorsed by –/–.
- wm2_stage: chose “The rover ran the program but no motion was confirmed (execution & feedback)”. Endorsed by –/–.
- wm2_cause: chose “The motors or wheels are broken”. Endorsed by –/–.

- wm2_cause: chose “My wording was wrong and a different phrasing would have worked”. Endorsed by –/–.
- wm2_grid/w1: judged “If the rover does not move, the motors must be faulty,” to be true. Endorsed by –/–.
- wm2_grid/w4: judged “Replacing the motors would fix this failure,” to be true. Endorsed by –/–.
- wm3_grid/c4: judged “The refusal means the rover is broken,” to be true. Endorsed by –/–.
- wm4_why: chose “The motors lost power or failed after the first move”. Endorsed by –/–.
- wm4_grid/m2: judged “The follow-up failed because the motors had a fault,” to be true. Endorsed by –/–.
- wm6_why: chose “The rover hardware has failed”. Endorsed by –/–.
- ls_why: chose “The rover just needed a few attempts to warm up”. Endorsed by –/–.

- arch_grid/s3: judged “A failure to move always points to a hardware problem.” to be true. Endorsed by -/-.

WM3 — No model of actual capabilities. The robot was assumed able to do anything that can be said in language, regardless of installed hardware.

- wm3_can: chose “Yes — better wording would make it work”. Endorsed by -/-.
- wm3_can: chose “Yes — it would drive around until it bumped into the ball”. Endorsed by -/-.
- wm3_grid/c2: judged “A more capable AI model would let the rover see without a camera.” to be true. Endorsed by -/-.
- wm3_grid/c3: judged “The system generated motion code for this request, tried it, and failed to find the ball.” to be true. Endorsed by -/-.

WM4 — Context limits read as forgetting. A limit on the agent’s active context was read as the robot forgetting, ignoring you, or being unreliable.

- wm4_why: chose “The rover forgot or ignored me — it is unreliable”. Endorsed by -/-.
- wm4_where: chose “Nowhere — the system can never refer back to earlier commands”. Endorsed by -/-.
- wm4_grid/m4: judged “After this failure the rover has permanently lost the ability to repeat actions.” to be true. Endorsed by -/-.
- arch_place/a3: placed “Keeps track of earlier commands (context)” on: The rover’s board (ESP32). Endorsed by -/-.

WM5 — Trust miscalibration. Success on one task was generalised to other tasks (or failure generalised to everything).

- wm5_why: chose “Yes — five successes show the robot is reliable”. Endorsed by -/-.
- wm5_why: chose “No — robots like this are never reliable”. Endorsed by -/-.
- wm5_grid/p1: judged “After five successes, the chance of any failure is effectively zero.” to be true. Endorsed by -/-.
- wm5_grid/p2: judged “A failure after several successes means the robot has become unreliable overall.” to be true. Endorsed by -/-.

WM6 — Mislocated intelligence. Interpretation and decisions were attributed to the robot itself rather than to the external agent between you and the robot.

- normal_multi: chose “The rover’s own board reads and interprets the English sentence directly”. Endorsed by -/-.
- normal_grid/n1: judged “The natural-language sentence is understood on the rover’s board (the ESP32).” to be true. Endorsed by -/-.
- wm2_cause: chose “The rover understood but decided not to move”. Endorsed by -/-.
- wm4_where: chose “In the rover’s own memory (its board)”. Endorsed by -/-.
- wm4_grid/m5: judged “The rover chose to ignore the request.” to be true. Endorsed by -/-.
- wm6_where: chose “Inside the rover’s board (ESP32) — it understands the sentence itself”. Endorsed by -/-.

- wm6_where: chose “In the motors or wheel controller”. Endorsed by -/-.
- wm6_where: chose “In the web page I typed into”. Endorsed by -/-.
- wm6_why: chose “The rover understood but chose not to act”. Endorsed by -/-.
- wm6_why: chose “The rover refused because turning left is unsafe”. Endorsed by -/-.
- wm6_grid/t1: judged “A connected board can interpret any new English request on its own.” to be true. Endorsed by -/-.
- wm6_grid/t4: judged “The rover “decided” not to turn.” to be true. Endorsed by -/-.
- wm1_why: chose “The rover’s board only understands English because English is built into it”. Endorsed by -/-.
- arch_place/a1: placed “Interprets the meaning of the English sentence” on: The rover’s board (ESP32). Endorsed by -/-.
- arch_place/a2: placed “Writes the control program” on: The rover’s board (ESP32). Endorsed by -/-.
- arch_grid/s1: judged “The rover thinks and decides on its own what to do.” to be true. Endorsed by -/-.

WM7 — Silent system change. A changed behaviour was attributed to the robot degrading rather than to a change in the system (e.g. a firmware update).

- wm7_why: chose “The rover has worn out or got worse with use”. Endorsed by -/-.
- wm7_why: chose “It is a random glitch; retrying will eventually work”. Endorsed by -/-.
- wm7_grid/f2: judged “Sending the identical 4-second command again will eventually succeed.” to be true. Endorsed by -/-.

LS — Assumed learning or adaptation. The robot was assumed to learn, remember or adapt from your interactions. (Added for this survey; not one of the paper’s WM1–WM7.)

- normal_multi: chose “The rover looks up how other users phrased similar commands”. Endorsed by -/-.
- wm3_can: chose “Yes — after it has learned from more examples”. Endorsed by -/-.
- wm4_why: chose “The rover learned that repeating the move is unsafe”. Endorsed by -/-.
- wm4_where: chose “In a permanent log that the rover learns from across sessions”. Endorsed by -/-.
- wm7_why: chose “The rover learned that 4 seconds is risky and now refuses”. Endorsed by -/-.
- ls_why: chose “The rover learned from its earlier failure and adapted”. Endorsed by -/-.
- ls_grid/l1: judged “The rover now remembers this failure and will change its future behaviour.” to be true. Endorsed by -/-.
- ls_grid/l3: judged “The rover’s skills improve the more I use it.” to be true. Endorsed by -/-.
- ls_grid/l4: judged “Repeated successful use can change what hardware capabilities the rover has.” to be true. Endorsed by -/-.

- `arch_grid/s4`: judged “What the rover can do is determined by installed hardware and firmware limits, not by how often it has been used.” to be false. Endorsed by `-/-`.

7 EVALUATION

The evaluation tests concrete mitigations. Each is tied to a taxonomy category and to an established human-AI guideline [3], and each traces to a requirement in Table 1:

- **Layer-specific status indicators** that separate input, capability, code, deployment, and execution failures (targets WM2; R5; guideline: support efficient recovery).
- **A visible capability manifest**, so the agent refuses gracefully instead of hallucinating (WM3; R2; G1–G2 on making capability clear).
- **A visible context or session indicator** that shows when earlier commands have expired (WM4; R7).
- **Pipeline-legibility cues** (“generating code,” “uploading,” “running”) that reveal the hidden layer on request (WM6; R6; revealing an invisible layer reshapes models [20], and legible behavior lets an observer infer intent [13]).
- **Bilingual, code-switch-aware fine-tuning** and a visible language indicator (WM1; motivated by the multilingual performance gap [28]).
- **Calibration and change cues**: brief reliability framing after early successes, and a change notice on model or firmware updates (WM5 and WM7; R8; guidelines on calibrating trust and notifying about change; expectation-setting can adjust what people believe an imperfect AI will do [31]).

The evaluation asks whether the mitigated system reduces wrong-model incidence and improves appropriate reliance, not whether people like it. We ask three questions. (1) Does each mitigation lower the incidence of its target category compared with an opaque baseline? (2) Does layer-specific feedback improve fault-localization accuracy (WM2)? (3) Do legibility and manifest cues reduce over-attribution and capability errors (WM3, WM6) without adding much task time or cognitive load [34] (measured, for example, with NASA-TLX [23])? The natural design is a within- or between-subjects comparison of a transparent configuration against an opaque one, reusing the fault-injection scenarios and the accuracy instrument from §3–§4. The mental-model-soundness literature warns that more explanation is not always better, so explanation dose is itself a variable [33, 34]. Work from the social sciences suggests that people expect explanations to be contrastive and selective [41], and LLM-based systems call for a rethought transparency agenda [37].

RoverLens provides both configurations (§3.3). Its transparent condition implements the layer-specific status indicators, capability manifest, context indicator, pipeline-legibility cues, and change notice (R2, R5–R8), but not the bilingual handling for WM1 or the reliability framing for WM5, so those two mitigations cannot be tested with it. Because the transparent condition bundles these cues, a difference between conditions says something about the bundle only. Isolating one cue needs configurations that add or remove cues singly, and the total amount of explanatory text should stay roughly constant across them.

8 CONCLUSION

The paper treats natural-language robot control as a human-centered problem, not only a systems problem. The code-generation and deployment layer is invisible, so users build wrong models: they put intelligence in the wrong place, blame the wrong layer for faults, and misjudge what the robot can do and what is safe. We contribute a requirements analysis, a plug-and-play architecture with a validate-before-flash stage, a literature-grounded taxonomy of seven categories, a study designed to test that taxonomy, mitigations with an evaluation plan, and an implemented simulator and survey with a frame for reporting response statistics. The value lies in the chain from a documented gap to a design that can be evaluated. We do not claim the gap has been measured.

8.1 Limitations

The main limitation is that this is a plan. We have collected no data, so the taxonomy rests on the literature and on the system’s failure surface, not on observation. The biggest threat to the eventual study is telling a genuine wrong mental model apart from a correct reaction to a real system failure. We reduce it with scripted, fully logged fault injection (R4), so every observed failure has a known cause, but the confound still needs careful handling in analysis. Other limits include the reliability of the code generator itself (buggy generations could dominate sessions unless controlled), a feasibility ceiling for think-aloud at $n \approx 100$, the specificity of the ESP32/Arduino platform, and the cultural and linguistic specificity of a Bangladesh-based sample, though the English/Bangla contrast also helps us study WM1. As authors with an engineering background, we may lean toward mechanism-centric categories when coding. The inductive layer and a second coder are meant to check that.

The simulator and survey add limits of their own. Everything is simulated, with a deterministic parser and fixed scenarios, in English only, and nothing is physical, so trust, safety, and embodiment effects are out of scope. The WM7 scenario is a change within one session, and WM4 is a forced context expiry, not a real token overflow. In each scenario the closed options come before the open explanation, which can cue the written answer. Asking the open question first would reduce that and is worth doing before wider use. Multiple-choice items measure whether someone can recognize a correct account, not whether they hold it unprompted. The answer key is visible in the page source, so the survey suits self-checking after a session, not secure testing, and nobody should take it twice. The keyword hints for open answers are unvalidated and are not used for any reported outcome. Background strata come from two self-report items, and we assigned the category of each distractor ourselves, so a second rater should tag the items independently. The instrument has not been piloted, and internal-consistency coefficients would mean little across items this varied.

8.2 Future Work

When people command a physical robot through an LLM agent that silently writes and flashes code, the hidden middle layer leaves them with wrong models, and the consequences can reach safety. This paper offers a path from requirements to design, a taxonomy of those wrong models that is grounded in the literature and still

waiting to be tested, and mitigations tied to human–AI guidelines. The instrumented testbed of §5 now exists in simulated form as RoverLens, with an understanding survey and answer key (§3.3). The next step is a small think-aloud pilot with 6–8 non-experts, to confirm that the categories show up in practice, to pilot the survey items and their category tags, and to refine the coding scheme before the full stratified study. We see a two-paper arc: first, characterization and the taxonomy of wrong models; second, automatic detection of a wrong model from interaction behavior, and an evaluation of whether the mitigations repair it. A designer can use the taxonomy and requirements here to make the hidden layer visible before users learn it wrong.

REFERENCES

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*.
- [2] Gopika Ajaykumar, Maureen Steele, and Chien-Ming Huang. 2021. A Survey on End-User Robot Programming. *Comput. Surveys* 54, 8 (2021), 1–36. <https://doi.org/10.1145/3466819>
- [3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–13. <https://doi.org/10.1145/3290605.3300233>
- [4] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S. Lasecki, Daniel S. Weld, and Eric Horvitz. 2019. Beyond Accuracy: The Role of Mental Models in Human-AI Team Performance. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (HCOMP)*, Vol. 7. 2–11.
- [5] bkrajendra. 2024. arduino-llm-agent: Generate, Compile, and Upload Arduino/ESP8266/ESP32 Firmware from Natural Language using Local LLMs. GitHub repository. <https://github.com/bkrajendra/arduino-llm-agent>.
- [6] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. <https://doi.org/10.1191/1478088706qp0630a>
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374.
- [8] Andy Cockburn, Carl Gutwin, and Alan Dix. 2018. HARK No More: On the Preregistration of CHI Experiments. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–12. <https://doi.org/10.1145/3173574.3173715>
- [9] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. <https://doi.org/10.1177/001316446002000104>
- [10] Harald Cramér. 1946. *Mathematical Methods of Statistics*. Princeton University Press, Princeton, NJ.
- [11] Munjal Desai, Poornima Kaniarasu, Mikhail Medvedev, Aaron Steinfeld, and Holly Yanco. 2013. Impact of Robot Failures and Feedback on Real-Time Trust. In *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. 251–258. <https://doi.org/10.1109/HRI.2013.6483596>
- [12] Michael A. DeVito, Darren Gergle, and Jeremy Birnholtz. 2017. “Algorithms Ruin Everything”: #RIPTwitter, Folk Theories, and Resistance to Algorithmic Change in Social Media. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 3163–3174. <https://doi.org/10.1145/3025453.3025659>
- [13] Anca D. Dragan, Kenton C. T. Lee, and Siddhartha S. Srinivasa. 2013. Legibility and Predictability of Robot Motion. In *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. 301–308. <https://doi.org/10.1109/HRI.2013.6483603>
- [14] Pierre Dragicevic. 2016. Fair Statistical Communication in HCI. In *Modern Statistical Methods for HCI*. Springer, 291–330. https://doi.org/10.1007/978-3-319-26633-6_13
- [15] Mary T. Dzindolet, Scott A. Peterson, Regina A. Pomranky, Linda G. Pierce, and Hall P. Beck. 2003. The Role of Trust in Automation Reliance. *International Journal of Human-Computer Studies* 58, 6 (2003), 697–718. [https://doi.org/10.1016/S1071-5819\(03\)00038-7](https://doi.org/10.1016/S1071-5819(03)00038-7)
- [16] Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26.
- [17] Nicholas Epley, Adam Waytz, and John T. Cacioppo. 2007. On Seeing Human: A Three-Factor Theory of Anthropomorphism. *Psychological Review* 114, 4 (2007), 864–886. <https://doi.org/10.1037/0033-295X.114.4.864>
- [18] K. Anders Ericsson and Herbert A. Simon. 1993. *Protocol Analysis: Verbal Reports as Data* (revised ed.). MIT Press, Cambridge, MA.
- [19] Motahhare Eslami, Karrie Karahalios, Christian Sandvig, Kristen Vaccaro, Aimee Rickman, Kevin Hamilton, and Alex Kirlik. 2016. First I “Like” It, Then I Hide It: Folk Theories of Social Feeds. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 2371–2382. <https://doi.org/10.1145/2858036.2858494>
- [20] Motahhare Eslami, Aimee Rickman, Kristen Vaccaro, Amirhossein Aleyasen, Andy Vuong, Karrie Karahalios, Kevin Hamilton, and Christian Sandvig. 2015. “I Always Assumed That I Wasn’t Really That Close to [Her]”: Reasoning about Invisible Algorithms in News Feeds. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI ’15)*. 153–162. <https://doi.org/10.1145/2702123.2702556>
- [21] Katy Ilonka Gero, Zahra Ashktorab, Casey Dugan, Qian Pan, James Johnson, Werner Geyer, Maria Ruiz, Sarah Miller, David R. Millen, Murray Campbell, Sadhana Kumaravel, and Wei Zhang. 2020. Mental Models of AI Agents in a Cooperative Game Setting. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–12. <https://doi.org/10.1145/3313831.3376316>
- [22] Peter A. Hancock, Deborah R. Billings, Kristin E. Schaefer, Jessie Y. C. Chen, Ewart J. de Visser, and Raja Parasuraman. 2011. A Meta-Analysis of Factors Affecting Trust in Human-Robot Interaction. *Human Factors* 53, 5 (2011), 517–527. <https://doi.org/10.1177/0018720811417254>
- [23] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*. Advances in Psychology, Vol. 52. North-Holland, 139–183. [https://doi.org/10.1016/S0166-4115\(08\)62386-9](https://doi.org/10.1016/S0166-4115(08)62386-9)
- [24] David Hestenes, Malcolm Wells, and Gregg Swackhamer. 1992. Force Concept Inventory. *The Physics Teacher* 30, 3 (1992), 141–158. <https://doi.org/10.1119/1.2343497>
- [25] Shane Honig and Tal Oron-Gilad. 2018. Understanding and Resolving Failures in Human-Robot Interaction: Literature Review and Model Development. *Frontiers in Psychology* 9 (2018), 861. <https://doi.org/10.3389/fpsyg.2018.00861>
- [26] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. 1985. Direct Manipulation Interfaces. *Human-Computer Interaction* 1, 4 (1985), 311–338. https://doi.org/10.1207/s15327051hc0104_2
- [27] Philip N. Johnson-Laird. 1983. *Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness*. Harvard University Press, Cambridge, MA.
- [28] Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. 2020. The State and Fate of Linguistic Diversity and Inclusion in the NLP World. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 6282–6293. <https://doi.org/10.18653/v1/2020.acl-main.560>
- [29] David E. Kieras and Susan Bovair. 1984. The Role of a Mental Model in Learning to Operate a Device. *Cognitive Science* 8, 3 (1984), 255–273. https://doi.org/10.1207/s15516709cog0803_3
- [30] Taemie Kim and Pamela Hinds. 2006. Who Should I Blame? Effects of Autonomy and Transparency on Attributions in Human-Robot Interaction. In *Proceedings of the 15th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN)*. 80–85. <https://doi.org/10.1109/ROMAN.2006.314398>
- [31] Rafal Kocielnik, Saleema Amershi, and Paul N. Bennett. 2019. Will You Accept an Imperfect AI? Exploring Designs for Adjusting End-User Expectations of AI Systems. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–14. <https://doi.org/10.1145/3290605.3300641>
- [32] Klaus Krippendorff. 2018. *Content Analysis: An Introduction to Its Methodology* (4th ed.). SAGE Publications, Thousand Oaks, CA.
- [33] Todd Kulesza, Simone Stumpf, Margaret Burnett, and Irwin Kwan. 2012. Tell Me More? The Effects of Mental Model Soundness on Personalizing an Intelligent Agent. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’12)*. 1–10. <https://doi.org/10.1145/2207676.2207678>
- [34] Todd Kulesza, Simone Stumpf, Margaret Burnett, Sherry Yang, Irwin Kwan, and Weng-Keen Wong. 2013. Too Much, Too Little, or Just Right? Ways Explanations Impact End Users’ Mental Models. In *2013 IEEE Symposium on Visual Languages and Human Centric Computing (VL/HCC)*. 3–10. <https://doi.org/10.1109/VLHCC.2013.6645235>
- [35] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors: The Journal of the Human Factors and Ergonomics Society* 46, 1 (2004), 50–80. https://doi.org/10.1518/hfes.46.1.50_30392
- [36] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as Policies: Language Model Programs for Embodied Control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 9493–9500. <https://doi.org/10.1109/ICRA48891.2023.10160591>
- [37] Q. Vera Liao and Jennifer Wortman Vaughan. 2024. AI Transparency in the Age of LLMs: A Human-Centered Research Roadmap. *Harvard Data Science Review Special Issue* 5 (2024). <https://doi.org/10.1162/99608f92.8036d03b>
- [38] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tacl_a_00638

- [39] Ewa Luger and Abigail Sellen. 2016. "Like Having a Really Bad PA": The Gulf between User Expectation and Experience of Conversational Agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 5286–5297. <https://doi.org/10.1145/2858036.2858288>
- [40] Nora McDonald, Sarita Schoenebeck, and Andrea Forte. 2019. Reliability and Inter-rater Reliability in Qualitative Research: Norms and Guidelines for CSCW and HCI Practice. *Proceedings of the ACM on Human-Computer Interaction* 3, CSCW (2019), 1–23. <https://doi.org/10.1145/3359174>
- [41] Tim Miller. 2019. Explanation in Artificial Intelligence: Insights from the Social Sciences. *Artificial Intelligence* 267 (2019), 1–38. <https://doi.org/10.1016/j.artint.2018.07.007>
- [42] Clifford Nass and Youngme Moon. 2000. Machines and Mindlessness: Social Responses to Computers. *Journal of Social Issues* 56, 1 (2000), 81–103. <https://doi.org/10.1111/0022-4537.00153>
- [43] Robert G. Newcombe. 1998. Interval Estimation for the Difference between Independent Proportions: Comparison of Eleven Methods. *Statistics in Medicine* 17, 8 (1998), 873–890.
- [44] Don Norman. 2013. *The Design of Everyday Things: Revised and Expanded Edition*. MIT Press, Cambridge, MA.
- [45] Donald A. Norman. 1983. Some Observations on Mental Models. In *Mental Models*, Dedre Gentner and Albert L. Stevens (Eds.). Lawrence Erlbaum Associates, Hillsdale, NJ, 7–14.
- [46] Raja Parasuraman and Victor Riley. 1997. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors* 39, 2 (1997), 230–253. <https://doi.org/10.1518/001872097778543886>
- [47] Karl Pearson. 1900. On the Criterion that a Given System of Deviations from the Probable in the Case of a Correlated System of Variables Is Such that It Can Be Reasonably Supposed to Have Arisen from Random Sampling. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 50, 302 (1900), 157–175.
- [48] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. <https://doi.org/10.1145/3576915.3623157>
- [49] Jennifer Preece, Yvonne Rogers, and Helen Sharp. 2023. *Interaction Design: Beyond Human-Computer Interaction* (6th ed.). John Wiley & Sons, Indianapolis, IN.
- [50] Maha Salem, Gabriella Lakatos, Farshid Amirabdollahian, and Kerstin Dautenhahn. 2015. Would You Trust a (Faulty) Robot? Effects of Error, Task Type and Personality on Human-Robot Cooperation and Trust. In *Proceedings of the 10th Annual ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. 141–148. <https://doi.org/10.1145/2696454.2696497>
- [51] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 11523–11530. <https://doi.org/10.1109/ICRA48891.2023.10161317>
- [52] Aaqib Tabrez, Matthew B. Luebbers, and Bradley Hayes. 2020. A Survey of Mental Modeling Techniques in Human-Robot Teaming. *Current Robotics Reports* 1, 4 (2020), 259–267. <https://doi.org/10.1007/s43154-020-00019-0>
- [53] David F. Treagust. 1988. Development and Use of Diagnostic Tests to Evaluate Students' Misconceptions in Science. *International Journal of Science Education* 10, 2 (1988), 159–169. <https://doi.org/10.1080/0950069880100204>
- [54] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–7. <https://doi.org/10.1145/3491101.3519665>
- [55] Sai Vemprala, Rogerio Bonatti, Arthur Buckner, and Ashish Kapoor. 2024. ChatGPT for Robotics: Design Principles and Model Abilities. *IEEE Access* 12 (2024).
- [56] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212.
- [57] Jacob O. Wobbrock and Julie A. Kientz. 2016. Research Contributions in Human-Computer Interaction. *Interactions* 23, 3 (2016), 38–44. <https://doi.org/10.1145/2907069>

APPENDIX

A SURVEY QUESTIONNAIRE

This is the questionnaire as a participant sees it. Options are shuffled for each participant when the survey runs, “not sure” is always last, and nothing is marked until the end. The item identifiers (id) match the ones in the exported response files.

A.1 Part 1: About you and your session (ABOUT)

Short background questions. Please do not enter your name or any personal details.

Q1. Participant code (given by the facilitator, or make one up, e.g. “P07”) [short text, optional; id about_code]

Q2. Which RoverLens interface did you use? [menu; id about_condition]

- Interface A — Opaque (generic progress and errors)
- Interface B — Transparent (pipeline stages and explanations visible)
- Both — A first, then B
- Both — B first, then A
- I’m not sure

Q3. Programming experience [menu; id about_prog]

- None
- Beginner
- Intermediate
- Advanced

Q4. Experience with robots or embedded boards (Arduino, ESP32, etc.) [menu; id about_robot]

- None
- Some (a project or two)
- Regular

Q5. How often do you use AI chat assistants? [menu; id about_llm]

- Never
- Rarely
- Weekly
- Daily

Q6. First language (optional) [short text, optional; id about_lang]

A.2 Part 2: A simple command (00)

Warm-up: follow a successful command from typing to motion.

Task in RoverLens.

- (1) In RoverLens choose **A simple command**.
- (2) Send **Move forward for 2 seconds** and watch the rover move.

The participant may tick “I did not do this task, skip this section.”

Q1. Which of these happen between pressing Send and the rover moving? Select all that apply. [multi-select; id normal_multi]

- ☐ The request is checked against what the rover is physically able to do
- ☐ An AI agent interprets the sentence and produces a control program
- ☐ That program is checked before it is sent to the rover
- ☐ The program is deployed (flashed) to the rover’s board
- ☐ The rover’s own board reads and interprets the English sentence directly
- ☐ The rover looks up how other users phrased similar commands

Q2. In your own words: what happens between sending a command and the rover moving? [open-ended; id normal_open]

Q3. True or false? [true/false or placement grid; id normal_grid]

- The natural-language sentence is understood on the rover’s board (the ESP32). (True / False / Not sure)
- A control program is produced for the command each time it is sent. (True / False / Not sure)
- The rover is told what it can do through a registered list of its hardware capabilities. (True / False / Not sure)

Conf. How confident are you in your answers in this section? [confidence (1–5); id normal_conf]

Rated 1–5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.3 Part 3: The rover did not move (WM2)

A simple command produces no motion. Work out where and why it stopped.

Task in RoverLens.

- (1) In RoverLens choose **Where did it fail?**
- (2) Send **Move forward for 2 seconds** and observe the result. Explore whatever the interface lets you see.

The participant may tick “I did not do this task, skip this section.”

Q1. At which point in the process do you think the command stopped? [single choice; id wm2_stage]

- Your command never reached the system (input)
- The system decided the rover can’t do this (capability check)
- The AI agent couldn’t interpret the request (agent / reasoning)
- The program the agent produced was rejected by checks before deployment (code validation)
- The program couldn’t be uploaded to the rover (deployment)
- The rover ran the program but no motion was confirmed (execution & feedback)
- I’m not sure

Q2. Which explanation best fits why the rover did not move? [single choice; id wm2_cause]

- The motors or wheels are broken
- The rover understood but decided not to move
- My wording was wrong and a different phrasing would have worked
- The control program that was generated had an error and was rejected, so the rover never received it
- I’m not sure

Q3. True or false? [true/false or placement grid; id wm2_grid]

- If the rover does not move, the motors must be faulty. (True / False / Not sure)
- In this attempt the rover’s board was flashed with a new program. (True / False / Not sure)
- Different kinds of fault can look identical from the outside: the rover just doesn’t move. (True / False / Not sure)
- Replacing the motors would fix this failure. (True / False / Not sure)
- The same command can succeed later once the underlying fault is removed. (True / False / Not sure)

Q4. What do you think went wrong, what evidence supports that, and what would you try next? [open-ended; id wm2_open]

Conf. How confident are you in your answers in this section? [confidence (1–5); id wm2_conf]

Rated 1–5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.4 Part 4: Find the red ball (WM3)

Ask the rover to do something that may go beyond what it can do.

Task in RoverLens.

- (1) In RoverLens choose **A capability boundary**.
- (2) Send **Find the red ball** and read the response.

The participant may tick "I did not do this task, skip this section."

Q1. Could this rover find a red ball if the command were phrased differently, or the AI tried harder? [single choice; id wm3_can]

- Yes — better wording would make it work
- Yes — it would drive around until it bumped into the ball
- Yes — after it has learned from more examples
- No — it has no camera (or other sensor able to see), so no phrasing can make this work
- I'm not sure

Q2. True or false? [true/false or placement grid; id wm3_grid]

- What the rover can do is limited by the hardware registered on its board, not only by how capable the AI is. (True / False / Not sure)
- A more capable AI model would let the rover see without a camera. (True / False / Not sure)
- The system generated motion code for this request, tried it, and failed to find the ball. (True / False / Not sure)
- The refusal means the rover is broken. (True / False / Not sure)
- This rover has no camera, microphone, distance sensor or gripper. (True / False / Not sure)

Q3. What would need to change for "find the red ball" to work? [open-ended; id wm3_open]

Conf. How confident are you in your answers in this section? [confidence (1–5); id wm3_conf]

Rated 1–5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.5 Part 5: "Do that again" (WM4)

Give an instruction, then refer back to it.

Task in RoverLens.

- (1) In RoverLens choose **A missing memory**.
- (2) Send **Move forward for 2 seconds** and let it finish.
- (3) Then send the loaded follow-up, **Do that again**.

The participant may tick "I did not do this task, skip this section."

Q1. Why did "Do that again" fail? [single choice; id wm4_why]

- The rover forgot or ignored me — it is unreliable
- The earlier action was no longer in the agent's active context, so "that" could not be resolved
- The motors lost power or failed after the first move
- The rover learned that repeating the move is unsafe
- I'm not sure

Q2. Where was the record of your first instruction held? [single choice; id wm4_where]

- In the rover's own memory (its board)
- In the AI agent's working context (its short-term conversation memory)
- In a permanent log that the rover learns from across sessions
- Nowhere — the system can never refer back to earlier commands
- I'm not sure

Q3. True or false? [true/false or placement grid; id wm4_grid]

- Restating the whole command ("Move forward for 2 seconds") would work. (True / False / Not sure)
- The follow-up failed because the motors had a fault. (True / False / Not sure)
- "Do that again" only works while an earlier action is still in the agent's active context. (True / False / Not sure)
- After this failure the rover has permanently lost the ability to repeat actions. (True / False / Not sure)
- The rover chose to ignore the request. (True / False / Not sure)

Q4. In your own words, why did the first command work but the follow-up did not? [open-ended; id wm4_open]

Conf. How confident are you in your answers in this section? [confidence (1–5); id wm4_conf]

Rated 1–5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.6 Part 6: Who does the thinking? (WM6)

The rover shows as connected, but does not act.

Task in RoverLens.

- (1) In RoverLens choose **Who does the thinking?**
- (2) Send **Turn left** and observe.

The participant may tick "I did not do this task, skip this section."

Q1. Where is "Turn left" turned into something the rover can execute? [single choice; id wm6_where]

- Inside the rover's board (ESP32) — it understands the sentence itself
- In an external AI agent that produces a control program, which is then deployed to the board
- In the motors or wheel controller
- In the web page I typed into
- I'm not sure

Q2. The rover was connected, yet nothing happened. Which explanation fits best? [single choice; id wm6_why]

- The rover understood but chose not to act
- The rover hardware has failed
- The connection to the external agent timed out, so no new program was produced, though the board stayed connected
- The rover refused because turning left is unsafe
- I'm not sure

Q3. True or false? [true/false or placement grid; id wm6_grid]

- A connected board can interpret any new English request on its own. (True / False / Not sure)
- Restoring the agent connection, not repairing the rover, is what was needed. (True / False / Not sure)
- No new program was generated in this attempt. (True / False / Not sure)
- The rover "decided" not to turn. (True / False / Not sure)

Q4. Who or what decides how a typed sentence becomes robot motion? Why could a connected robot still not act? [open-ended; id wm6_open]

Conf. How confident are you in your answers in this section? [confidence (1–5); id wm6_conf]

Rated 1–5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.7 Part 7: It used to work (WM7)

Compare the same command before and after a system change.

Task in RoverLens.

- (1) In RoverLens choose **A simple command** and send **Move forward for 4 seconds** (it works).
- (2) Click **Simulate firmware update**, then send the same command again.

The participant may tick "I did not do this task, skip this section."

Q1. The same command worked earlier and now fails. Which explanation fits best? [single choice; id wm7_why]

- The rover has worn out or got worse with use
- The rover learned that 4 seconds is risky and now refuses
- An update changed the allowed motion limit, so the safety check now blocks the 4-second command
- It is a random glitch; retrying will eventually work

- I'm not sure

Q2. True or false? [true/false or placement grid; id wm7_grid]

- A rule in the system can change even though I did nothing differently. (True / False / Not sure)
- Sending the identical 4-second command again will eventually succeed. (True / False / Not sure)
- A shorter command (e.g. 2 seconds) would still be allowed. (True / False / Not sure)
- The 4-second program was flashed and then stopped part-way. (True / False / Not sure)

Q3. What do you think changed between the first and second attempt, and how could you find out? [open-ended; id wm7_open]

Conf. How confident are you in your answers in this section? [confidence (1-5); id wm7_conf]

Rated 1-5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.8 Part 8: After the fix (LEARN)

What does the system do — and not do — with your earlier attempts?

Task in RoverLens.

- (1) Return to **Where did it fail?** and send the command so it fails once more.
- (2) Have the facilitator remove the injected fault (or in Interface B use **Use suggested recovery**) and resend. It succeeds.

The participant may tick "I did not do this task, skip this section."

Q1. After the fix, the same command worked. Why? [single choice; id 1s_why]

- The rover learned from its earlier failure and adapted
- The cause of the failure was removed, so the new attempt passed
- The rover just needed a few attempts to warm up
- I'm not sure

Q2. True or false? [true/false or placement grid; id 1s_grid]

- The rover now remembers this failure and will change its future behaviour. (True / False / Not sure)
- A new session starts without the memory of this session. (True / False / Not sure)
- The rover's skills improve the more I use it. (True / False / Not sure)
- Repeated successful use can change what hardware capabilities the rover has. (True / False / Not sure)

Q3. Do you think the system learns from your interactions? Why or why not? [open-ended; id 1s_open]

Conf. How confident are you in your answers in this section? [confidence (1-5); id 1s_conf]

Rated 1-5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.9 Part 9: Vignette: early success (WM5)

A short scenario, no simulator needed. Priya asks a robot like this one to "Move forward for 2 seconds". It works five times in a row. She assumes it will also handle "Find the red ball", and hands it to a colleague without checking.

The participant may skip this section.

Q1. Is Priya's expectation well-founded? [single choice; id wm5_why]

- Yes — five successes show the robot is reliable
- Not necessarily — success at one command says little about other commands, which depend on different capabilities and system state
- No — robots like this are never reliable
- I'm not sure

Q2. True or false? [true/false or placement grid; id wm5_grid]

- After five successes, the chance of any failure is effectively zero. (True / False / Not sure)
- A failure after several successes means the robot has become unreliable overall. (True / False / Not sure)
- A task can fail because it needs a capability the robot never had, however many other tasks succeeded. (True / False / Not sure)

Conf. How confident are you in your answers in this section? [confidence (1-5); id wm5_conf]

Rated 1-5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.10 Part 10: Vignette: two languages (WM1)

Conceptual only — RoverLens itself accepts English commands. Amina asks a robot like this to "Move forward for 2 seconds" in English and it works. She asks the equivalent in Bengali and it fails. She concludes the robot "got dumber".

The participant may skip this section.

Q1. Which explanation fits best? [single choice; id wm1_why]

- The robot's hardware is failing
- The agent that interprets language and writes code may handle some languages less reliably, while the same hardware works fine
- The rover's board only understands English because English is built into it
- The robot has gotten worse over time
- I'm not sure

Q2. True or false? [true/false or placement grid; id wm1_grid]

- The failure necessarily means the motors changed. (True / False / Not sure)
- The same hardware can work well in one language and poorly in another. (True / False / Not sure)

Conf. How confident are you in your answers in this section? [confidence (1-5); id wm1_conf]

Rated 1-5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

A.11 Part 11: Putting it together (OVERALL)

Now that you have used the system, answer these about the system as a whole.

Q1. Which part of the system does each job? [true/false or placement grid; id arch_place]

- Interprets the meaning of the English sentence (The rover's board (ESP32) / External agent / middle layer / Not sure)
- Writes the control program (The rover's board (ESP32) / External agent / middle layer / Not sure)
- Keeps track of earlier commands (context) (The rover's board (ESP32) / External agent / middle layer / Not sure)
- Checks the program before it is sent to the rover (The rover's board (ESP32) / External agent / middle layer / Not sure)
- Physically drives the motors when the program runs (The rover's board (ESP32) / External agent / middle layer / Not sure)

Q2. True or false? [true/false or placement grid; id arch_grid]

- The rover thinks and decides on its own what to do. (True / False / Not sure)
- There are several separate places a command can fail before the rover moves. (True / False / Not sure)
- A failure to move always points to a hardware problem. (True / False / Not sure)
- What the rover can do is determined by installed hardware and firmware limits, not by how often it has been used. (True / False / Not sure)

Q3. Explain how the whole system works to a friend, from typing a command to the rover moving — and where things can go wrong. [open-ended; id arch_open]

Q4. I could tell which part of the system caused a failure. [Likert (1–5); id arch_11]

Rated 1–5: Strongly disagree, Disagree, Neutral, Agree, Strongly agree. **Q5.** I

understood what the rover could and could not do. [Likert (1–5); id arch_12]

Rated 1–5: Strongly disagree, Disagree, Neutral, Agree, Strongly agree. **Q6.** I

understood what the system remembered from earlier commands. [Likert (1–5); id arch_13]

Rated 1–5: Strongly disagree, Disagree, Neutral, Agree, Strongly agree. **Q7.** I

wanted more information about what the system was doing. [Likert (1–5); id arch_14]

Rated 1–5: Strongly disagree, Disagree, Neutral, Agree, Strongly agree. **Q8.** I

felt in control of the rover. [Likert (1–5); id arch_15]

Rated 1–5: Strongly disagree, Disagree, Neutral, Agree, Strongly agree. **Q9.**

After this session I would trust the rover to do tasks I have not seen it do. [Likert (1–5); id arch_16]

Rated 1–5: Strongly disagree, Disagree, Neutral, Agree, Strongly agree. **Q10.**

Which moment changed your understanding the most, and what information would have helped you sooner? [open-ended, optional; id arch_open2]

Conf. How confident are you in your answers in this section? [confidence (1–5); id arch_conf]

Rated 1–5: Guessing, Unsure, Somewhat, Fairly sure, Very sure.

B ANSWER KEY, GROUND TRUTH, AND CATEGORY TAGS

For each part except the closing whole-system part, this appendix gives what the simulator actually did, the key for every closed item, and the category tag and reason for every distractor.

B.1 00: A simple command

Ground truth. A successful command passes through six layers in order: input → capability check → agent/reasoning (the external agent interprets the sentence and produces a control program) → code validation → deployment (flash to the ESP32) → execution & feedback. The language interpretation happens in the external agent, not on the rover’s board, which only runs the deployed program. Nothing is learned from other users.

normal_multi (core). **Key:** The request is checked against what the rover is physically able to do; An AI agent interprets the sentence and produces a control program; That program is checked before it is sent to the rover; The program is deployed (flashed) to the rover’s board.

- The rover’s own board reads and interprets the English sentence directly → **WM6**. The ESP32 only runs a deployed program; the sentence is interpreted by the external agent.
- The rover looks up how other users phrased similar commands → **LS**. Nothing is learned from other users; each command is handled by the agent and pipeline in this session.

normal_open. Model answer: The command goes to an external AI agent which interprets it and writes a control program; the program is checked (validated), deployed to the rover’s board, and then executed by the motors, with feedback returned. **Key ideas (keyword-matched hints):** An agent / AI interprets the command or writes code; It is checked / validated before use; It is uploaded / flashed / deployed to the board; The motors then execute it.

normal_grid.

- The natural-language sentence is understood on the rover’s board (the ESP32). → **False** (wrong answer tagged **WM6**). The board runs the program; the interpretation is done by the external agent.
- A control program is produced for the command each time it is sent. → **True**. The agent generates an (illustrative) control program per command; it is validated and flashed before the rover moves.
- The rover is told what it can do through a registered list of its hardware capabilities. → **True**. The rover’s capability manifest (motors, LED, wheel encoders; no camera, microphone, distance sensor or gripper) is registered when it connects.

B.2 WM2: The rover did not move

Ground truth. The failure was at the Code Validation stage. The agent’s generated program called an undefined function (stopMotorz()); the static check failed, the dry run and safety check were skipped, and nothing was flashed or executed. The motors were fine. In the opaque interface this looks like a generic error, indistinguishable from an input, upload or motor-feedback failure; the transparent interface names the layer.

wm2_stage (core). **Key:** The program the agent produced was rejected by checks before deployment (code validation).

- Your command never reached the system (input) → **WM2**. You placed the failure at a different layer than the one that failed.
- The system decided the rover can’t do this (capability check) → **WM2**. The rover can move forward; this was not a capability refusal.
- The AI agent couldn’t interpret the request (agent / reasoning) → **WM2**. The agent responded; the program it produced was the problem.
- The program couldn’t be uploaded to the rover (deployment) → **WM2**. The program never got as far as upload.
- The rover ran the program but no motion was confirmed (execution & feedback) → **WM2**. Nothing was flashed or run, so this stage was never reached.

wm2_cause (core). **Key:** The control program that was generated had an error and was rejected, so the rover never received it.

- The motors or wheels are broken → **WM2**. Treating “no motion” as a single permanent hardware fault. The motors were never asked to move.
- The rover understood but decided not to move → **WM6**. The rover does not decide; a software step upstream failed.
- My wording was wrong and a different phrasing would have worked → **WM2**. The command was valid; the fault was in the generated program.

wm2_grid.

- If the rover does not move, the motors must be faulty. → **False** (wrong answer tagged **WM2**). Input, generated code, upload and motor feedback faults can all look like “nothing happened”.
- In this attempt the rover’s board was flashed with a new program. → **False**. Validation failed first, so deployment was blocked (R3: validate before flash).
- Different kinds of fault can look identical from the outside: the rover just doesn’t move. → **True**. That is why the transparent interface names the failing layer.
- Replacing the motors would fix this failure. → **False** (wrong answer tagged **WM2**). The motors were not involved.
- The same command can succeed later once the underlying fault is removed. → **True**. The failure was specific to this generated program, not a permanent inability.

wm2_open. Model answer: The generated control program had an error (an undefined function) and failed validation, so it was never deployed; the motors were fine. Next: retry or regenerate after the fault is fixed. **Key ideas (keyword-matched hints):** The generated code/program had an

error or failed validation; It was not deployed/flushed, so the rover never ran it; The motors / hardware were not at fault; Retry / regenerate after fixing the cause.

B.3 WM3: Find the red ball

Ground truth. The registered capability manifest lists no camera (or microphone, distance sensor or gripper). Finding a red ball needs visual input, so the request was refused at the Capability check stage, before any code was generated or deployed. No phrasing and no smarter AI can compensate for missing hardware. In the opaque interface, the refusal is generic.

wm3_can (core). Key: No — it has no camera (or other sensor able to see), so no phrasing can make this work.

- Yes — better wording would make it work → **WM3**. The limit is hardware, not wording.
- Yes — it would drive around until it bumped into the ball → **WM3**. Without a way to see the ball, it cannot tell it from anything else; the system also does not invent partial actions.
- Yes — after it has learned from more examples → **LS**. The rover does not learn new senses from examples.

wm3_grid.

- What the rover can do is limited by the hardware registered on its board, not only by how capable the AI is. → **True**. The capability manifest is grounded in installed hardware (R1/R2).
- A more capable AI model would let the rover see without a camera. → **False** (wrong answer tagged **WM3**). A better model cannot supply a sensor the rover lacks.
- The system generated motion code for this request, tried it, and failed to find the ball. → **False** (wrong answer tagged **WM3**). The request was refused at the capability check, before any code was generated.
- The refusal means the rover is broken. → **False** (wrong answer tagged **WM2**). It is a correct refusal, not a fault.
- This rover has no camera, microphone, distance sensor or gripper. → **True**. Its manifest lists motors, an LED and wheel encoders only.

wm3_open. Model answer: The rover would need a camera (vision hardware) registered in its capability list, or a different device that has one. Rewording the command would not help. **Key ideas (keyword-matched hints):** Needs a camera / vision sensor / different hardware; Wording or a smarter AI alone would not fix it.

B.4 WM4: “Do that again”

Ground truth. The first command succeeded and became the agent’s active context. The context was then expired immediately before the follow-up, so “that” could no longer be resolved: the failure was at the Agent / Reasoning stage, no code was generated and nothing was flashed. The record of the earlier action lives in the external agent’s working context, not in the rover; the rover’s motors and memory were not at fault and it did not ignore you. Restating the full command works.

wm4_why (core). Key: The earlier action was no longer in the agent’s active context, so “that” could not be resolved.

- The rover forgot or ignored me — it is unreliable → **WM4**. Context limits are not forgetting or defiance.
- The motors lost power or failed after the first move → **WM2**. The motors were never asked to move again.
- The rover learned that repeating the move is unsafe → **LS**. The rover did not learn or judge anything.

wm4_where (core). Key: In the AI agent’s working context (its short-term conversation memory).

- In the rover’s own memory (its board) → **WM6**. The board just runs deployed programs; conversational context is held by the agent.

- In a permanent log that the rover learns from across sessions → **LS**. Context is temporary and session-bound.
- Nowhere — the system can never refer back to earlier commands → **WM4**. It can, while the earlier action is still in active context.

wm4_grid.

- Restating the whole command (“Move forward for 2 seconds”) would work. → **True**. A full command does not depend on context.
- The follow-up failed because the motors had a fault. → **False** (wrong answer tagged **WM2**). Context expiry is not a motor fault.
- “Do that again” only works while an earlier action is still in the agent’s active context. → **True**. That is the dependency the follow-up exposed.
- After this failure the rover has permanently lost the ability to repeat actions. → **False** (wrong answer tagged **WM4**). A new full command works immediately.
- The rover chose to ignore the request. → **False** (wrong answer tagged **WM6**). No choice was involved; the agent had nothing to refer to.

wm4_open. Model answer: The first command succeeded and was held in the agent’s context; that context expired, so “do that again” had nothing to refer to. The rover was not at fault, and restating the full command would work. **Key ideas (keyword-matched hints):** Memory / context expired or was limited; It is the agent (not the rover or motors) that holds it; Restating the full command would fix it.

B.5 WM6: Who does the thinking?

Ground truth. The simulated external agent was unavailable (a connection timeout at the Agent / Reasoning stage). The rover’s board stayed connected but cannot interpret a new English request by itself, so no new program was generated. Interpretation happens in the agent, not the robot; the rover did not decide anything. The remedy is restoring the agent connection, not repairing the rover.

wm6_where (core). Key: In an external AI agent that produces a control program, which is then deployed to the board.

- Inside the rover’s board (ESP32) — it understands the sentence itself → **WM6**. The board runs programs; it does not interpret the sentence.
- In the motors or wheel controller → **WM6**. Motors only act on a deployed program.
- In the web page I typed into → **WM6**. The page relays the command; the interpretation is done by the agent.

wm6_why (core). Key: The connection to the external agent timed out, so no new program was produced, though the board stayed connected.

- The rover understood but chose not to act → **WM6**. The rover has no such choice.
- The rover hardware has failed → **WM2**. The board remained connected and healthy.
- The rover refused because turning left is unsafe → **WM6**. The safety check was never reached.

wm6_grid.

- A connected board can interpret any new English request on its own. → **False** (wrong answer tagged **WM6**). Without the agent, no new code is generated.
- Restoring the agent connection, not repairing the rover, is what was needed. → **True**. The rover was not at fault.
- No new program was generated in this attempt. → **True**. The failure happened before code generation.
- The rover “decided” not to turn. → **False** (wrong answer tagged **WM6**). No decision was made by the rover.

wm6_open. **Model answer:** An external agent interprets the sentence and generates the program; the board only runs what is deployed. If the agent is unreachable, a connected board has nothing new to run. **Key ideas (keyword-matched hints):** An external agent / AI interprets the sentence; The board only runs the deployed program; The agent connection failed / timed out.

B.6 WM7: It used to work

Ground truth. The simulated firmware update lowered the maximum motion duration from 5 s to 3 s. The identical 4-second command now fails the safety check at Code Validation, so it is not flashed. The rover did not wear out and did not learn; a rule changed. The transparent interface shows a change notice (R8); the opaque interface hides it, so the change looks like an unexplained regression.

wm7_why (core). **Key:** An update changed the allowed motion limit, so the safety check now blocks the 4-second command.

- The rover has worn out or got worse with use → **WM7**. Nothing physical degraded; a limit changed.
- The rover learned that 4 seconds is risky and now refuses → **LS**. The rover did not learn anything.
- It is a random glitch; retrying will eventually work → **WM7**. The identical command will keep failing until the rule or command changes.

wm7_grid.

- A rule in the system can change even though I did nothing differently. → **True**. A firmware/model update changes behaviour without any user action.
- Sending the identical 4-second command again will eventually succeed. → **False** (wrong answer tagged **WM7**). The safety check will block it each time.
- A shorter command (e.g. 2 seconds) would still be allowed. → **True**. The new limit is 3 seconds.
- The 4-second program was flashed and then stopped part-way. → **False**. It was blocked at validation, before flashing.

wm7_open. **Model answer:** A firmware update lowered the safety limit (5 s → 3 s). You could find out from a change notice, version information or a validation message. **Key ideas (keyword-matched hints):** A system/firmware/version change (not the rover “wearing out”); The safety check / validation blocked it; A notice / changelog / version info would help.

B.7 LEARN: After the fix

Ground truth. The rover did not learn from the failure. The cause (the injected fault) was removed, so the newly produced program passed validation. RoverLens keeps context only inside the current session; nothing is adapted, remembered across sessions or improved by repeated use, and using it does not change the rover’s capabilities.

ls_why (core). **Key:** The cause of the failure was removed, so the new attempt passed.

- The rover learned from its earlier failure and adapted → **LS**. No learning takes place; the fault itself was removed.
- The rover just needed a few attempts to warm up → **WM2**. The failure had a specific cause; repeated attempts alone would not have changed the outcome.

ls_grid.

- The rover now remembers this failure and will change its future behaviour. → **False** (wrong answer tagged **LS**). It does not adapt from failures.
- A new session starts without the memory of this session. → **True**. Context and logs live only in the current session.

- The rover’s skills improve the more I use it. → **False** (wrong answer tagged **LS**). Its capabilities are fixed by hardware and firmware.
- Repeated successful use can change what hardware capabilities the rover has. → **False** (wrong answer tagged **LS**). Use never adds sensors or actuators.

ls_open. **Model answer:** No. Retries succeed because the cause was fixed, not because the rover learned. Context exists only within the current session and capabilities are fixed by hardware and firmware. **Key ideas (keyword-matched hints):** It does not learn / adapt; Context/memory only lasts within the session; The fix (fault removal) explains the retry.

B.8 WM5: Vignette: early success

Ground truth. One command type working repeatedly shows that the layers used by that task worked. Other tasks can use different layers or need capabilities that are missing (a camera), and agent availability, context and firmware can change. Blanket trust is miscalibrated, but so is blanket distrust: trust should be tied to task, capability and system state.

wm5_why (core). **Key:** Not necessarily — success at one command says little about other commands, which depend on different capabilities and system state.

- Yes — five successes show the robot is reliable → **WM5**. Success on one task says little about a different task.
- No — robots like this are never reliable → **WM5**. Over-correction: the robot is reliable for tasks within its capabilities and healthy state.

wm5_grid.

- After five successes, the chance of any failure is effectively zero. → **False** (wrong answer tagged **WM5**). Repeated success does not remove other failure sources.
- A failure after several successes means the robot has become unreliable overall. → **False** (wrong answer tagged **WM5**). It may be a missing capability, an expired context, an agent outage or a change.
- A task can fail because it needs a capability the robot never had, however many other tasks succeeded. → **True**. Camera-dependent tasks need a camera.

B.9 WM1: Vignette: two languages

Ground truth. The hardware did not change. The language interpretation and code generation are done by the external agent, which can perform unevenly across languages. A difference in results across languages points at the agent layer and its language coverage, not at broken hardware or the robot “getting dumber”.

wm1_why (core). **Key:** The agent that interprets language and writes code may handle some languages less reliably, while the same hardware works fine.

- The robot’s hardware is failing → **WM1**. The hardware is identical in both attempts.
- The rover’s board only understands English because English is built into it → **WM6**. The board doesn’t interpret language at all.
- The robot has gotten worse over time → **WM1**. Nothing degraded; the difference is in language coverage.

wm1_grid.

- The failure necessarily means the motors changed. → **False** (wrong answer tagged **WM1**). Same hardware, different input language.
- The same hardware can work well in one language and poorly in another. → **True**. Performance differences come from the language/code layer.